REP-3: Customer Behavior and Loyalty Report

Instructions for Ben (Part 1, storefront and onboarding)

Prepared 2026-11-04 from the TA's Checkpoint 3 feedback and the current code. Size L. Owner: Ben.

Item	Details
Your report	Today's **Rewards report** at **/staff/reports/rewards** (managers only). It lists points, tier, orders and spend per customer, all time. You replace it with a broader **Customer behavior and loyalty** report on the same route.
What you add	Purchase frequency, average spending, repeat customer rate, new versus returning customers, retention, purchasing preferences, rewards redemption rate (honest version, see section 2), and whether loyalty members buy more often or spend more than non-members. Plus filters, KPI cards, charts and a 'What this means' panel.
Files you create	`db/reports/customer_behavior.sql`, `db/reports/test_customer_behavior.sql`, `lib/customerMetrics.js`, `lib/customerPeriods.js`, `server/reports/customers.js`, tests under `tests/`.
Files you change	`src/app/pages/reports/RewardsReportPage.jsx` (rewritten; keep its default export) and one line in `src/app/routes.jsx` (the nav label and summary).
Files you must leave alone	`server/reports/rewards.js` (stays so its tests pass; your page may call it for the loyalty section), `db/core/schema.sql` (frozen), `db/core/views.sql`, `package.json` (REP-7 installs the chart library).
Depends on	Nothing to start. REP-0 (demo data) gives meaningful numbers; REP-7 gives charts and shared filter components. Until then build the data, endpoint, KPI cards and insights, then add charts.
Done when	Every metric above shows a number or chart on the demo data, the members versus non-members comparison states its definition, and the filters (date range, tier, segment) change the figures.

Important. SQL in this guide is a hand-checked starting point. It was written against `schema.sql` and follows the style of the existing views, but could not be run here (no SQL Server in the assistant's workspace). Run it on your local SQL Server Express first. Never run anything against the hosted Azure database yourself: the lead loads new SQL there (INT-1).

1. Why this report has to change

The TA's overall comment on all reports was: *"Your reports should go beyond showing what happened and help management understand why it happened, whether performance is improving or declining, and what actions the business should take. Include more meaningful calculations, comparisons, trends, visuals and flexible filters."*

For the rewards report the TA asked for a broader **Customer Behavior and Loyalty Analysis**: purchase frequency, average spending, repeat customer rate, new versus returning customers, retention, purchasing preferences, rewards redemption rate, and whether loyalty members buy more often or spend more than non-members.

What exists today versus what the TA asked for

TA expectation	Today	What you add
Purchase frequency and average spending	Orders and total spent per customer, all time, in a table.	Orders per customer per month, average order value and spend per customer, for a chosen period, with the change against the previous one.
Repeat customer rate	Not shown.	Share of customers with two or more orders, and the share of orders that come from repeat buyers.
New versus returning customers	Not shown.	Monthly split of customers and revenue into new (first order that month) and returning.
Retention	Not shown.	Share of new customers who buy again within 30, 60 and 90 days, and a cohort table by first-order month.
Purchasing preferences	Not shown.	Top categories by revenue and by customers, per tier and per segment.
Do members buy more than non-members?	Not shown. Only a tier column.	Side-by-side comparison with a written definition of 'member'.
Rewards redemption rate	Points are shown as earned and outstanding only.	An honest panel: points cannot be redeemed today, so the page says so and shows the unredeemed balance (see below).
Explain why and what to do	Management must work it out.	A 'What this means' panel: 'Only 31% of customers have bought twice: offer a bonus on the second order', 'N valuable customers are at risk'.

The point of the change: today a manager sees *who has points*. After the change a manager sees *who comes back, who is slipping away, what each group buys, and whether the loyalty program changes behavior*.

2. The data you will use and its limits

Table	Columns you need	Used for
SALES_TRANSACTION	Transaction_id, Customer_id, Transaction_date, Subtotal, Status ('open', 'completed', 'voided')	Only Status = 'completed' counts as an order.
TRANSACTION_ITEM	Transaction_id, Product_id, Quantity, Line_total	Units per order; revenue per category.
PRODUCT, CATEGORY	Product_id, Category_id, Name	Purchasing preferences.
CUSTOMER	Customer_id, Created_date, Loyalty_points, Tier_id	Customer since, current tier.
REWARD_TIER, REWARD_PROGRAM	Tier_name (Bronze, Silver, Gold), Min_points; Points_per_dollar, Redemption_rate	Tier filter and member definition.
LOYALTY_TRANSACTION	Customer_id, Points_change, Reason, Created_at	Points earned, taken back, outstanding.
PRODUCT_RETURN / _ITEM	Status 'refunded', Product_id, Quantity	Optional: refunded revenue per order.

Rule. Every query and every join must filter **Marked_for_deletion = 0**, including the lookup tables. The application login cannot DELETE rows; the project uses soft delete everywhere.

Limits you must respect and print on the page

- **Redemption cannot be measured yet.** Points are earned only; nothing spends them, and `REWARD_PROGRAM.Redemption_rate` is unused. A redemption rate would always read 0%. Show the panel title 'Rewards redemption' with the text **'Redemption is not offered yet'**, report the outstanding points as 'unredeemed balance', and write the endpoint so the formula (points redeemed / points earned) switches on by itself if LOY-1 ever adds redemption rows. Do not invent a number.
- **Every customer earns points,** so 'loyalty member' has no obvious meaning. Use this definition and print it on the page: **member = the customer currently has a tier (Bronze or higher); non-member = 'No tier'.**
- **The members comparison is biased.** Tiers are earned by spending, so heavy buyers become members. Members will always look better. Print: 'Members earn their tier by buying, so this shows association, not proof that the program changes behavior.'
- **No tier history.** The database stores the current tier only, so a customer is classified by today's tier for all their orders.
- **Revenue is before tax and before refunds,** like the existing views (`Subtotal`). Say so.
- **The seed is tiny** (7 customers, 8 sales). Retention and cohorts are empty until REP-0. Every panel needs an empty state.
- **Privacy:** the page is managers only. Return customer names where needed (as the old report did) but never passwords or emails you do not need.

3. New parameters

The new endpoint is GET /api/reports/customers (manager only). All parameters are optional, validated on the server, echoed back in the response and kept in the page URL so a filtered view can be shared.

Parameter	Values	Default	Effect and validation
from, to	YYYY-MM-DD	last 90 days	The period for orders, active customers and new customers. Real calendar date check (copy `realDate` from `server/reports/sales.js`), start not after end, range at most 400 days (HTTP 400).
compare	previous, lastyear, none	previous	previous = equal-length period immediately before `from`. lastyear = same dates one year earlier. none = no change figures.
tier	No tier, or a tier name from REWARD_TIER	all	Only customers who currently hold that tier. Check the value against the tier names read from the database; anything else: 400.
segment	champions, loyal, new, at_risk, dormant, one_time	all	Only customers in that segment as of `to` (rules in section 4). Anything else: 400.
category	A Category_id (integer)	all	Only orders containing that category (revenue then means revenue of that category).
groupBy	month, week	month	Buckets of the new versus returning chart and trends. Weeks start on Monday.
top	1 to 50	10	Length of the top customers list. Same validation as `rewards.js` (`validate.integer`).

Date presets (last 30 days, last 90 days, this quarter, year to date) are page buttons that fill `from` and `to`.

Coordination. REP-7 will later provide a shared `lib/reportPeriods.js`. Until it merges, put your own helper in `lib/customerPeriods.js` with names `previousPeriod`, `lastYearPeriod`, `buckets`. Terry (REP-2) does the same in his own file; the small duplication is deliberate so neither of you waits for the other. REP-7 replaces both.

4. Metrics and formulas

Fetch the data with a few simple queries and do the calculations in pure functions in `lib/customerMetrics.js` (not in `server/`, where every file becomes a Vercel route). The functions take plain rows, so you can test them with fixtures and no database. The endpoint reads **all completed orders up to the end date**, because new, returning, repeat and retention all depend on earlier history.

Metric	Formula	Notes
Active customers	Customers with at least one order in the period	After tier, segment and category filters.
Orders per customer per month	Orders in the period / active customers / (days in period / 30.44)	The purchase frequency. Null when no active customers.
Average order value	Revenue / orders	Revenue = Subtotal (after discounts, before tax).
Spend per customer	Revenue / active customers	Average spending in the period.
Repeat customer rate	Customers with 2 or more orders up to the end date / customers with 1 or more	Lifetime, as of the end date. Compare with the same figure as of the day before the start date.
Repeat order share	Orders that are not a customer's first / all orders	Order_number greater than 1.
New customer	First-ever order falls in the period (Order_number = 1)	Counted in the month or week of that order.
Returning customer	Ordered in the period and had an earlier order	
N-day retention (N = 30, 60, 90)	Among customers whose first order is in the period and at least N days before the end date: share with a second order within N days of the first	Excludes customers too recent to judge. Under 10 eligible customers: show 'not enough data'.
Cohort table	Rows: first-order month. Columns: month 0 to 5 after it. Cell: share of the cohort that ordered in that month	Leave future months blank.
Days between orders	Days_since_prev_order from the view	Show the average and a small histogram.
Revenue concentration	Share of revenue from the top 10% of active customers (at least 1)	Used by an insight.
Segments (first matching rule wins)	**New**: first order within 30 days of the end date. **Champions**: 3 or more orders and last order within 30 days. **Loyal**: 2 or more orders and last order within 60 days. **At risk**: 2 or more orders and last order 61 to 120 days ago. **Dormant**: last order over 120 days ago. **One-time**: everyone else with an order.	All thresholds are constants at the top of the library. Customers with no orders are counted separately as 'no purchases yet' and are not in any segment.
Member and non-member figures	The same four numbers (orders per customer per month, order value, spend per customer, repeat rate) for tier present versus 'No tier'	Also show the ratio (members / non-members). Print the definition and the bias warning.

Metric	Formula	Notes
Category preference	Revenue and customer count per category, per tier and per segment	From the category view. Show the top 3 per group as shares of that group's revenue.
Points earned, taken back, outstanding	Earned: points above zero in the period. Taken back: points below zero. Outstanding: sum of Loyalty_points	Same meanings as the existing report. Earned per active customer is a useful extra.
Redemption rate	Points redeemed / points earned	Only when redemption rows exist (see section 2). Otherwise return redemptionOffered = false.
Change (percent)	(current - previous) / previous x 100	Null when previous is 0 (show 'new'). Rates are compared in points.

5. Build procedure

Step 1. Create the views (db/reports/customer_behavior.sql)

A view cannot take parameters, so the views hold the joins and window functions; the endpoint adds the filters. Because the window functions run before the endpoint's date filter, `Order_number` is the customer's real order number over their whole history, which is what 'new' and 'returning' need.

```
-- VW_CUSTOMER_ORDERS: one row per completed order, with the customer's order number
CREATE OR ALTER VIEW VW_CUSTOMER_ORDERS AS
SELECT o.Transaction_id, o.Customer_id, o.Order_date, o.Subtotal,
       l.Units, l.Lines, l.Refunded_revenue,
       ROW_NUMBER() OVER (PARTITION BY o.Customer_id
                          ORDER BY o.Order_date, o.Transaction_id) AS Order_number,
       DATEDIFF(DAY, LAG(o.Order_date) OVER (PARTITION BY o.Customer_id
                          ORDER BY o.Order_date, o.Transaction_id), o.Order_date)
                                                                  AS Days_since_prev_order,
       cu.Created_date AS Customer_since,
       ISNULL(t.Tier_name, N'No tier') AS Tier_name
FROM  (SELECT s.Transaction_id, s.Customer_id, s.Subtotal,
              CAST(s.Transaction_date AS DATE) AS Order_date
       FROM   SALES_TRANSACTION s
       WHERE  s.Status = 'completed' AND s.Marked_for_deletion = 0) o
JOIN   CUSTOMER cu ON cu.Customer_id = o.Customer_id AND cu.Marked_for_deletion = 0
LEFT JOIN REWARD_TIER t ON t.Tier_id = cu.Tier_id AND t.Marked_for_deletion = 0
OUTER APPLY (
    SELECT SUM(ti.Quantity) AS Units, COUNT(*) AS Lines,
           CAST(SUM(ROUND(ti.Line_total * f.Qty / ti.Quantity, 2)) AS DECIMAL(12,2))
                                                                  AS Refunded_revenue
    FROM   TRANSACTION_ITEM ti
    CROSS APPLY (SELECT ISNULL(SUM(ri.Quantity), 0) AS Qty
                 FROM   PRODUCT_RETURN r
                 JOIN   PRODUCT_RETURN_ITEM ri ON ri.Return_id = r.Return_id
                 WHERE  r.Transaction_id = ti.Transaction_id
                   AND  ri.Product_id    = ti.Product_id
                   AND  r.Status = 'refunded'
                   AND  r.Marked_for_deletion = 0 AND ri.Marked_for_deletion = 0) f
    WHERE  ti.Transaction_id = o.Transaction_id AND ti.Marked_for_deletion = 0
) l;
GO

-- VW_CUSTOMER_ORDER_CATEGORIES: one row per order and category, for purchasing preferences
CREATE OR ALTER VIEW VW_CUSTOMER_ORDER_CATEGORIES AS
SELECT s.Transaction_id, s.Customer_id, CAST(s.Transaction_date AS DATE) AS Order_date,
       c.Category_id, c.Name AS Category_name,
       SUM(ti.Quantity) AS Units,
       CAST(SUM(ti.Line_total) AS DECIMAL(12,2)) AS Revenue
```

```
FROM   SALES_TRANSACTION s
JOIN   CUSTOMER cu ON cu.Customer_id = s.Customer_id AND cu.Marked_for_deletion = 0
JOIN   TRANSACTION_ITEM ti ON ti.Transaction_id = s.Transaction_id AND ti.Marked_for_deletion = 0
JOIN   PRODUCT p  ON p.Product_id  = ti.Product_id AND p.Marked_for_deletion = 0
JOIN   CATEGORY c ON c.Category_id = p.Category_id AND c.Marked_for_deletion = 0
WHERE  s.Status = 'completed' AND s.Marked_for_deletion = 0
GROUP BY s.Transaction_id, s.Customer_id, CAST(s.Transaction_date AS DATE), c.Category_id, c.Name;
GO
```

Check: run the file twice (it must not fail the second time). For one customer with three orders, `Order_number` must read 1, 2, 3 and `Days_since_prev_order` must be NULL, then the real gaps. `SUM(Subtotal)` over a customer must equal `Total_spent` in `VW_REWARDS_REPORT`.

> **Version.** LAG and ROW_NUMBER need SQL Server 2012 or newer and work on Azure SQL. If your local Express is older than that, tell the lead before changing the approach.

Step 2. Write a rolled-back test script (db/reports/test_customer_behavior.sql)

Follow the pattern of `verify.sql`: open a transaction, insert known rows, assert the views, then ROLLBACK so nothing is left behind. Cover at least:

- A customer with three completed orders on known dates: Order_number 1, 2, 3 and the exact day gaps.
- An 'open' or 'voided' order and a flagged order or flagged customer: not in the view, and they do not shift Order_number.
- An order with items in two categories: two rows in the category view, revenue adds up to the order's Subtotal.
- A customer with no tier: Tier_name = 'No tier'.
- A partly refunded order: Refunded_revenue is the proportional amount.

Step 3. Write the pure metric and period modules

Function	Input	Output
previousPeriod, lastYearPeriod, buckets	Dates and 'month' or 'week'	Same behaviour as described for REP-2: equal-length previous period, same dates last year, ordered buckets without gaps.
customerStats(orders, asOfDate)	All orders up to a date	Per customer: orders, spend, firstOrder, lastOrder, recencyDays, tier.
segmentOf(stats)	One customer's stats	'new', 'champions', 'loyal', 'at_risk', 'dormant' or 'one_time' (first matching rule).
periodKpis(orders, period, filters)	Orders and a period	activeCustomers, ordersPerCustomerPerMonth, avgOrderValue, spendPerCustomer, repeatRatePct, repeatOrderSharePct, newCustomers, returningCustomers.
newVsReturning(orders, buckets)	Orders and buckets	Per bucket: new and returning customers, and their revenue.
retention(orders, period, days)	Orders, period, 30 or 60 or 90	{ eligible, returned, ratePct } or { notEnoughData: true }.
cohortTable(orders, months)	Orders and a month list	Rows with cohort size and percent per month after (null for future months).
memberComparison(orders, period)	Orders and a period	{ members, nonMembers, ratios } with the four figures each.
categoryPreferences(rows, customerGroups)	Category rows and groups (tier or segment)	Top 3 categories per group with the share of that group's revenue.
buildCustomerInsights(kpis, previousKpis, segments, members, loyalty)	The pieces above	Up to 5 { id, severity, text, action }. See section 7.

Use integer cents when summing money so totals never drift by a cent.

Step 4. Write the endpoint (server/reports/customers.js)

1. Start with `requireAuth(req, ['manager'])` and the validation of section 3 (`HttpError(400, ...)` for bad values, `validate.integer` for numbers).
2. Read the tier names from `REWARD_TIER` (Marked_for_deletion = 0) to validate the `tier` parameter and to order tiers by `Min_points`.
3. Query `VW_CUSTOMER_ORDERS` for all orders with `Order_date <= @to` (named parameters only). Query `VW_CUSTOMER_ORDER_CATEGORIES` for the period (and the previous period when comparing).
4. Query `LOYALTY_TRANSACTION` for points earned and taken back in the period, whether any row has the reason 'Redemption', and the sum of `CUSTOMER.Loyalty_points` for the outstanding balance. Read `db/core` (the trigger and return procedures) to see the exact Reason texts used for earning and taking back, so refund take-backs are never mistaken for redemptions.
5. Compute everything with the library; apply `tier`, `segment` and `category` filters to customers and orders before calculating the figures.

6. Return: **filters, period, comparePeriod, kpis, newVsReturning, retention, cohorts, segments, members, preferences, topCustomers, loyalty (earned, takenBack, outstanding, tiers, redemptionOffered, redemptionRatePct), insights**.
7. Run **npm run routes** so `lib/routes.generated.js` includes the new endpoint, and commit it. `npm test` fails if the generated list is stale.

> **Keep.** Leave `server/reports/rewards.js` unchanged. Its tests keep passing, and the page can still call it for the tier mix and top earners, which already work.

Step 5. Rebuild the page (RewardsReportPage.jsx)

1. Change the nav label to 'Customer behavior and loyalty' and the summary text in `src/app/routes.jsx` (one line, the path stays `/staff/reports/rewards`). REP-7 also edits that file; keep your change to that single line to avoid a conflict, and tell the REP-7 owner.
2. Put new small components in `src/app/pages/reports/customers/` (filter bar, KPI card, segment table, cohort table, insights, member comparison). When REP-7 merges you replace them with the shared ones.
3. Layout, top to bottom: header, filter bar, KPI cards, 'What this means' panel, new versus returning chart, retention and cohort table, segments, members versus non-members, purchasing preferences, loyalty section (points and redemption), top customers table. The wireframe on the last page shows it.
4. Read the filters from the URL query string and write them back when they change.
5. Every KPI card shows its value, the change in words and colour, and the previous value in small text.
6. Loyalty section: reuse the existing numbers (points earned, taken back, outstanding, tier mix, top earners). The redemption panel says 'Redemption is not offered yet' and shows the unredeemed balance.
7. Charts: until REP-7 adds a chart library, draw bars with plain `div` elements and inline styles (allowed by the site's security policy). The cohort table can be a normal table with cell background shade set inline.
8. Segment names always come with a one-line meaning (hover or small text), and every colour is paired with words.
9. Empty states: no orders in the period; not enough data for retention; no customers in the chosen segment.

Step 6. Tests

Test file	What it checks	How
tests/lib/customerMetrics.test.js	Every formula of section 4 with fixtures: first order versus returning, repeat rate with one customer, retention cut-off for recent customers and the 10-customer minimum, cohort blanks, each segment rule and the order of rules, member ratios, previous period 0, insight rules.	Plain fixtures, no database.
tests/lib/customerPeriods.test.js	Previous period, last year (29 February), week and month buckets without gaps.	Plain fixtures.
tests/customersReport.test.js	Bad dates, tier, segment, category or top give 400; valid filters reach the query as named parameters; the redemption flag is false with no redemption rows; support 403; no login 401; SQL text contains Marked_for_deletion = 0.	Use `tests/helpers_manager.js` with `jest.mock('../lib/db', ...)` as in `tests/catalog.test.js`.

Do not use `helpers_backoffice.js` for filter tests: its request builder does not support query parameters.

6. Visual KPIs and chart ideas

The reader first sees the cards, then the meaning panel, then the evidence in charts. Each item says what question it answers; cut anything that does not answer one.

6.1 KPI cards

Card	Value	Change shown	Question it answers
Active customers	Customers who bought in the period	Percent change; new customers in small text	Is the customer base growing?
Orders per customer per month	Purchase frequency	Percent change	Do customers buy more often?
Average order value	Revenue / orders	Percent change	Do they spend more each time?
Repeat customer rate	Customers with 2 or more orders	Points up or down	Do first-time buyers come back?
90-day retention	New customers who bought again within 90 days	Points up or down	Are we keeping the customers we win?
Member spend lift	Member spend per customer / non-member	None (ratio shown with its definition)	Does the program go with higher spending?
Optional: spend per customer, new customers, revenue share of the top 10%	See section 4	As above	Who carries the revenue?

6.2 Charts

Chart	Type	What it shows
New versus returning customers	Stacked bars per month (new on the bottom), with the returning share as a line label	Whether growth comes from new buyers or loyal ones. The TA asked for it by name.
Retention cohort table	Heat table: first-order month rows, months-after columns, darker is higher	Whether customers still buy 1, 2, 3 months after the first order.
Segment mix	Donut of customers and a second donut of revenue	Where the value is: Champions and Loyal usually carry most revenue; At risk and Dormant are the win-back target.
Members versus non-members	Grouped bars: orders per month and average order value, each for the two groups	Whether members buy more often or spend more. Always next to the definition and the bias note.
Top categories per tier	Grouped or stacked bars: categories on the axis, one series per tier	What each tier prefers (purchasing preferences).
Purchase frequency distribution	Histogram of customers by number of orders (1, 2, 3, 4, 5+)	How many customers are one-time buyers.
Days between orders	Histogram	How long a typical return takes, so 'at risk' makes sense.
Points earned versus taken back	Line or bars per month	Loyalty program activity, including refunds clawing points back.
Revenue concentration	Cumulative curve (customers sorted by spend)	How dependent the store is on its best customers. Extra if time allows.

Minimum for 'done': new versus returning, the cohort or retention figures, the segment mix, and the members versus non-members comparison.

7. 'What this means' panel: the insight rules

These are rule-based sentences, not guesses. Each rule has a condition, a sentence built from real numbers and a recommended action. Show at most five, most severe first. Put the rules in `buildCustomerInsights` so they can be tested. Skip comparison rules when compare is none or the previous period is empty.

Id	Severity	Condition	Sentence and action
repeat	Warning	Repeat customer rate under 30%	"Only 31% of customers have bought more than once." Action: a bonus-points offer for the second order.
retention	Danger or Good	90-day retention changed by 5 points or more	"Fewer new customers come back: 90-day retention fell from 42% to 35%." Action: follow up after the first purchase.
newDrop	Warning	New customers fell 15% or more	"New customers fell 22%." Action: check promotion and storefront traffic.
atRisk	Danger	At risk segment has customers	"N customers who bought 2 or more times have not returned in 61 to 120 days; together they spent $X." Action: send them an offer (customers have notification settings).

Id	Severity	Condition	Sentence and action
dormant	Warning	Dormant share of customers over 40%	"Dormant customers are 45% of everyone who ever bought." Action: a win-back campaign.
members	Info	Both groups have at least 5 customers	"Members order 2.1 times as often and spend 38% more per order (member = has a tier)." Action: promote the program to non-members. Always add: association, not proof.
concentration	Warning	Top 10% of customers give over 50% of revenue	"The top 10% of customers bring 58% of revenue." Action: protect them (early access, perks).
preference	Info	A group's top category is over 40% of its revenue and differs from another group's	"Gold members spend 45% on Consoles; customers without a tier spend most on Accessories." Action: tailor offers to each group.
unredeemed	Info	Redemption not offered and outstanding points above 0	"N points are outstanding and none can be redeemed yet." Action: ask the lead whether to build redemption (LOY-1).
allgood	Good	No danger or warning above	"Customer behavior is stable compared with the previous period."

8. Traps to avoid

- **Counting only the period.** New, returning, repeat and retention need all earlier orders. Fetch history up to the end date, then filter.
- **Filtering orders before numbering them.** Order_number comes from the view over the whole history. Never recompute it after a date filter.
- **Open and voided sales.** Only completed sales are orders; the views filter that, so do not bypass them.
- **Soft delete.** A forgotten `Marked_for_deletion = 0` on a joined table brings back deleted customers or categories. Test it.
- **Young customers in retention.** A customer who first bought 10 days ago cannot have returned within 90 days yet. Exclude them from the denominator.
- **Tiny groups.** Under 5 or 10 customers a percentage is noise. Show 'not enough data'.
- **Overclaiming.** Members versus non-members shows association. The page must say so.
- **Fake redemption numbers.** Do not show 0% as a result. Say the feature is not offered.
- **Percent of zero.** A previous value of 0 gives 'new', never infinity or NaN. Rates change in points.
- **Date handling.** 'Today' is the server date (UTC on Azure). Acceptable; note it in the PR.
- **Colour alone.** Pair every colour with words.
- **Server files become routes.** Put helper code in `lib/`, not in `server/`.

9. Definition of done

[ ]	Both views are in `db/reports/customer_behavior.sql`
[ ]	The rolled-back test script passes and leaves no rows behind.
[ ]	One customer checked by hand: order numbers, day gaps, spend and tier match what the page shows.
[ ]	Pure functions in `lib/` have tests for every formula, each segment rule, retention cut-offs and the empty cases.
[ ]	The new endpoint is manager-only and tested (200, 403, 401, 400); every parameter is validated.
[ ]	`server/reports/rewards.js` is unchanged and its tests pass; `npm run routes` was run and the generated file committed; `npm test` passes.
[ ]	Every metric of section 4 shows a number or chart on the demo data: frequency, average spending, repeat rate, new versus returning, retention, preferences, members versus non-members, redemption panel.
[ ]	The members comparison prints its definition and the bias note; the redemption panel says 'Redemption is not offered yet'.
[ ]	The filters (date range, compare, tier, segment, category) change the figures; the page has KPI cards with changes, a meaning panel with 3 to 5 insights, charts, tables and empty states.
[ ]	The nav label reads 'Customer behavior and loyalty'; the route path is unchanged.
[ ]	The PR description lists the parameters, the new files and the manual checks you did.

Questions to ask the lead before you start: (1) Is 'member = has a tier' acceptable as the definition? Suggested: yes. (2) Is the 30/60/120-day segment scheme right for a game store? Suggested: yes, keep thresholds as constants. (3) Should LOY-1 (real redemption) be built? If yes, this report will show the true redemption rate with no further change.

Wireframe: how the Customer behavior and loyalty page could look

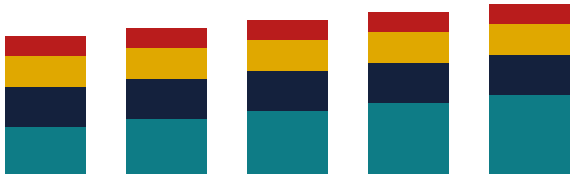
Customer behavior and loyalty

Aug 6 to Nov 3, 2026, compared with May 8 to Aug 5 (previous period)

Dates: Last 90 days | Compare: Previous period | Tier: All | Segment: All | Category: All | Group by: Month

184	0.82	31.0%	$149	35%	1.4x
Active customers	Orders / customer / month	Repeat customer rate	Average order value	90-day retention	Member spend lift
up 9%, 41 new	up 5%	down 3.0 pts	up 3%	down 7 pts	members vs non-members

New vs returning customers
monthly; new at the bottom



Retention by first-order month
darker = more came back

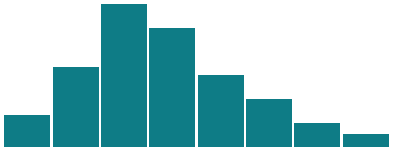
What this means

- 23 customers who bought 2+ times have not returned in 61 to 120 days ($4,800 spent). *Send them a win-back offer.*
- Only 31% of customers have bought twice. *Offer bonus points on the second order.*
- 90-day retention fell from 42% to 35%. *Follow up after the first purchase.*
- Members order 1.8x as often (member = has a tier). Association, not proof. *Promote the program to non-members.*
- Redemption is not offered yet; 41,200 points are unredeemed. *Decide on LOY-1.*

Segment mix
Champions, Loyal, At risk, Dormant

Members vs non-members
orders / month and order value

Orders per customer
1, 2, 3, 4, 5+ orders



Customer	Tier	Segment	Orders	Spend	Avg order	Last order	Days ago	Points

Reading order: filters, six KPI cards with changes, the 'What this means' panel (right), charts (left), then the tables: cohort details, members versus non-members with its definition, loyalty section, top customers. The numbers are examples only.

Honest labels: the members comparison always prints its definition and the 'association, not proof' note, and the redemption panel says 'Redemption is not offered yet'. Until REP-7 installs a chart library, draw bars with plain div elements and shade the cohort cells inline.